



SIMULACIONES Y PROCESOS ESTOCÁSTICOS

2026

Teoría: Dr. Alejandro Mezio – mezioalejandro@uca.edu.ar

Práctica: Dr. Luis Herrera – lherrera@uca.edu.ar

METAHEURÍSTICAS

- G. Ciaburro, Hand-os Simulations...: Capítulo 7 y 8

Límite de los métodos con gradiente

(Recordemos que queremos minimizar alguna función objetivo o función de pérdida)

Para calcular el gradiente se requiere:

- Que la función sea diferenciable (o al menos un poco suave)
- Que el espacio de búsqueda sea continuo
- Que los mínimos locales no sean un problema grave (una función convexa lo garantiza)

En la práctica, muchos problemas no cumplen estas condiciones:

- Problema del viajante (TSP, traveling salesman problem): espacio de búsqueda discreto y permutaciones
- Diseño de horarios, routing, scheduling: espacios combinatorios
- Optimización de hiperparámetros de ML: tenemos un espacio mixto (continuo + discreto)
- Funciones multimodales con muchos mínimos locales

Límite de los métodos con gradiente

¿Qué hacemos cuando el espacio de búsqueda es tan grande que no podemos optimizar exhaustivamente?

Muchos problemas reales requieren encontrar:

- la mejor decisión
- la mejor configuración
- la mejor solución

dentro de un espacio enorme de alternativas.

¿Cómo encontrar buenas soluciones cuando la búsqueda exhaustiva es imposible?

Metaheurísticas

Algoritmos avanzados de propósito general diseñados para resolver problemas complejos de optimización

Vienen en nuestra ayuda cuando encontrar la solución exacta tomaría demasiado tiempo (o es matemáticamente imposible)

No garantizan encontrar el óptimo global, pero tienen como objetivo encontrar soluciones lo suficientemente buenas con un costo computacional razonable

Principio fundamental → Balance entre:

- **Exploración**: explorar el espacio. Descubrir nuevas regiones del espacio de soluciones para evitar quedar atrapado en "óptimos locales"
- **Explotación**: refinar soluciones conocidas. Mejorar y ajustar detalladamente las soluciones prometedoras que ya se han encontrado.

Simulated Annealing (recocido)

Su nombre viene del proceso de recocido en la metalurgia. Consiste en:

- Calentar el metal a alta temperatura → átomos se pueden mover libremente (temperatura = energía cinética, de movimiento)
- Enfriar lentamente → los átomos se acomodan en configuraciones de baja energía
- Si el enfriamiento es rápido (quenching) → estado metaestable (mínimo local)
- Si el enfriamiento es lento → formación de cristal perfecto (mínimo global)

Para nuestro caso, hacemos la siguiente analogía:

- Energía del sistema → función objetivo o pérdida $f(\theta)$
- Temperatura → parámetro que controla la probabilidad de aceptar movimientos malos
- Enfriamiento → reducción gradual de esa probabilidad

Simulated Annealing (recocido)

Supongamos tenemos que minimizar una función pérdida $L(w)$ donde $w = (w_1, w_2, \dots, w_d)$,

Buscamos $w^* = \arg \min_w L(w)$

Iterativamente hacemos $w^{(n+1)} = w^{(n)} + \Delta w^{(n)}$, y hay varias maneras de obtener $\Delta w^{(n)}$

Simulated annealing se trata de buscar aleatoriamente un nuevo w vecino, si obtengo un menor $L(w)$ lo acepto, pero si obtengo mayor lo puedo también aceptar con una probabilidad dada por la temperatura y por el valor de cuánto empeora $L(w)$

Algoritmo:

- Inicializamos con un $w^{(0)}$, y una temperatura inicial $T = T_0$ (alta)
- Proponemos un nuevo $w^{(1)}$ vecino/cercano
- Calculamos $\Delta L = L(w^{(1)}) - L(w^{(0)})$ (aquí L hace el papel de la energía E)
- Si $\Delta L < 0$ (mejora): aceptamos el nuevo valor $w^{(1)}$
- Si $\Delta L > 0$ (empeora): aceptamos $w^{(1)}$ con una probabilidad $e^{-\frac{\Delta L}{T}}$
- Bajamos la temperatura $T \leftarrow \alpha \cdot T$ (normalmente $\alpha \sim 0.95 - 0.99$, velocidad enfriamiento)

Simulated Annealing (recocido)

- De esta manera, vamos explorando el espacio de w , pero no a lo pavote, sino con un criterio que balancea el “buscar el menor valor de L ” con “permitir movernos lo suficiente y evitar mínimos locales”

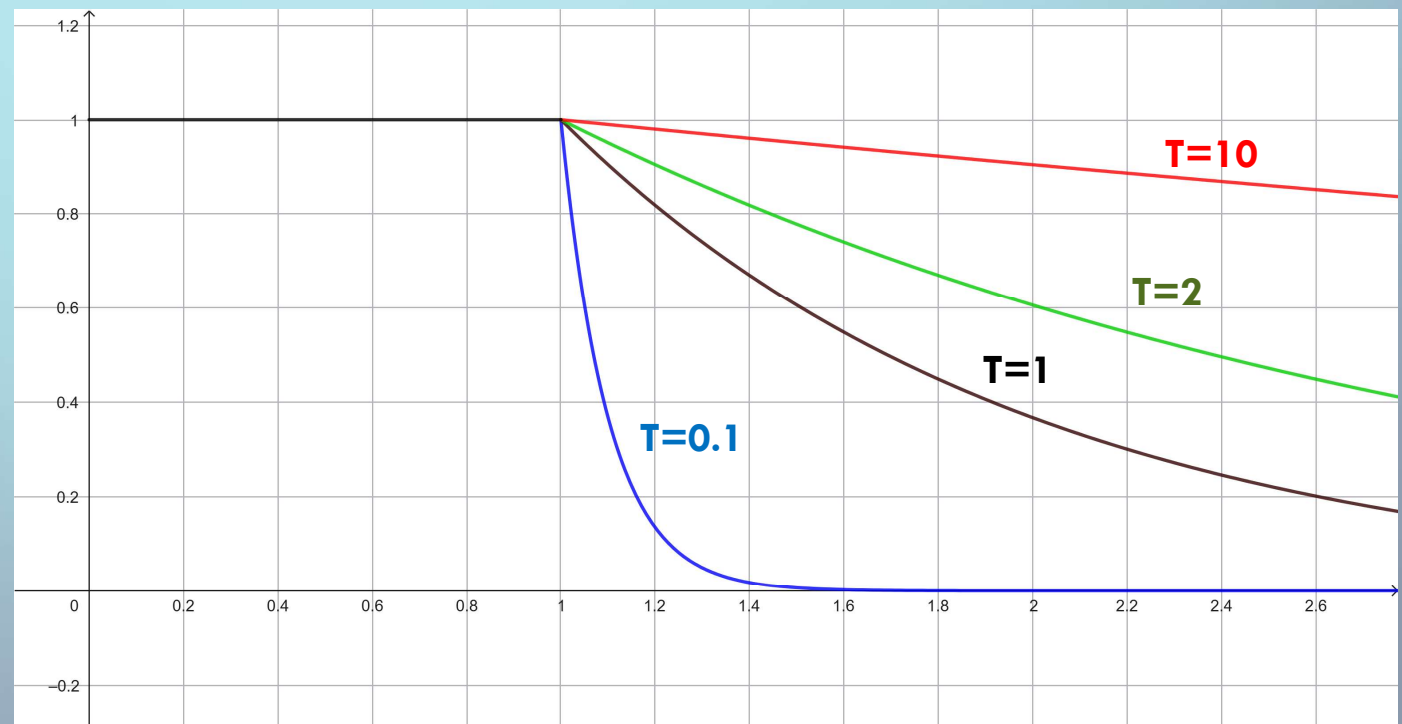
Veamos la función de probabilidad de aceptación:

- Tomamos $L(w^{(0)}) = 1$
- $T = 0.1, 1, 2, 10$

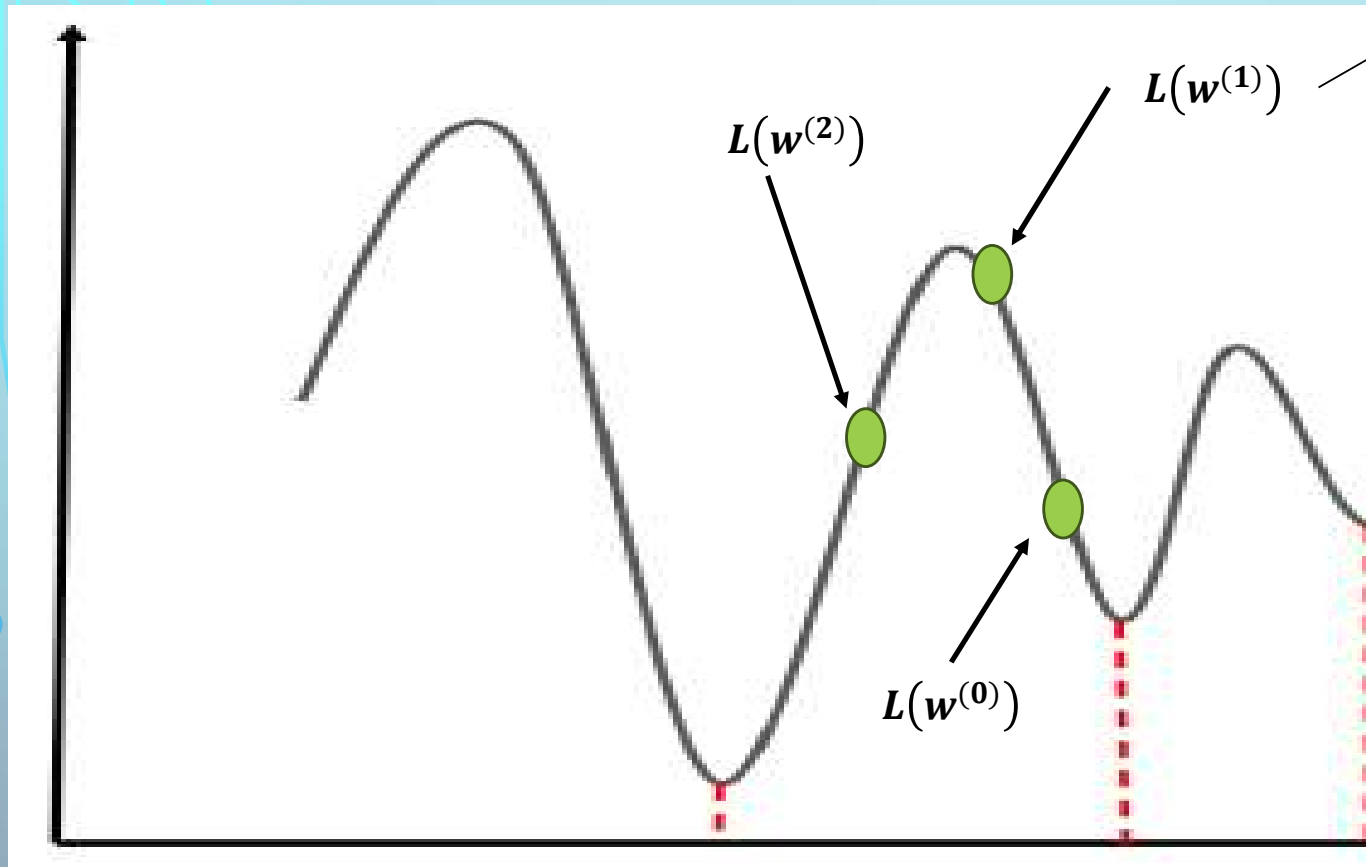
$$P(\text{aceptar}) = \begin{cases} 1, & L(w) < 1 \\ e^{-\Delta L/T}, & L(w) \geq 1 \end{cases}$$

Altas temperaturas: casi todo se acepta $\rightarrow$ exploración pura

Bajas temperaturas: se aceptan las mejores $\rightarrow$ explotación



Simulated Annealing (recocido)



Si la temperatura es lo suficientemente alta, podemos quedarnos con este punto, y así tendríamos más posibilidad de acceder al mínimo global

Simulated Annealing (recocido)

La velocidad de enfriamiento es crítica:

- Geométrica: $T(n) = \alpha^n \cdot T_0$ (más común)
- Logarítmica: $T(n) = \frac{T_0}{\ln(1+n)}$ (garantías teóricas de convergencia)
- Lineal: $T(n) = T_0 - c \cdot n$ (rara vez usado)

También es importante cómo elegimos los valores nuevos vecinos:

- Si el espacio w es continuo: $w^{(n+1)} = w^{(n)} + \varepsilon$, con $\varepsilon \sim N(0, \sigma^2)$
- Problema del viajante: permutar dos elementos (ciudades)
- Números binarios: cambiar un bit

VER NOTEBOOKS

Otros métodos

`scipy.optimize.minimize(..., method="método",...)`

método:

- Nelder-Mead
- Powell
- CG (Conjugate Gradient)
- BFGS
- Newton-CG

Algoritmos Genéticos

Los algoritmos genéticos se inspiran en:

- Evolución biológica
- Selección Natural

Darwin (1859): los individuos más aptos tienen mayor probabilidad de reproducirse y transmitir sus características

Idea conceptual:

- Las soluciones “compiten”
- Las mejores/aptas sobreviven y generan “descendencia”

Un Algoritmo Genético mantiene una población de soluciones

Algoritmos Genéticos

Traduciendo al lenguaje de optimización:

Biología	Algoritmo Genético
Individuo	Solución candidata w
Cromosoma / genes	Representación de w (vector de bits, etc)
Fitness (aptitud)	Valor de la función objetivo $L(w)$
Generación	Una iteración del algoritmo
Reproducción	Operador de cruce
Mutación	Perturbación aleatoria de genes
Selección Natural	Sesgo hacia individuos con mejor fitness

las soluciones mejoran generación a generación
porque las aptas se reproducen más

Ciclo:

1. Evaluar
2. Seleccionar
3. Recombinar
4. Mutar
5. Repetir

Algoritmos Genéticos

Primero de todo, debemos decidir cómo representar una solución:

- Representación binaria (la más clásica)

$$w = [1\ 0\ 1\ 1\ 0\ 0\ 1\ 0\ 1\ 1]$$

cada bit es un gen



- Representación Real (natural para optimización continua)

$$w = [3.14, -0.72, 1.05, 2.88]$$

gen1 gen2 gen3 gen4

- Representación de permutación (para problemas de secuenciación)

$$w = [3, 1, 4, 2, 5] \quad (\text{orden de ciudades en problema del viajante})$$

la representación define el espacio de búsqueda.
Una buena representación hace que soluciones similares tengan cromosomas similares (*principio de continuidad*)

Algoritmos Genéticos

Algoritmo general:

Núcleo de AG

1. INICIALIZACIÓN

Generar población $P = \{\theta_1, \theta_2, \dots, \theta_N\}$ aleatoriamente

2. EVALUACIÓN

Calcular fitness $f(\theta_i)$ para cada individuo

3. REPETIR hasta criterio de parada:

a. SELECCIÓN

Elegir pares de padres con probabilidad $\propto$ fitness

b. CRUCE (crossover)

Combinar cada par de padres $\rightarrow$ hijos

c. MUTACIÓN

Perturbar aleatoriamente algunos genes de los hijos

d. EVALUACIÓN

Calcular fitness de la nueva generación

e. REEMPLAZO

La nueva generación reemplaza a la anterior
(opcionalmente: conservar los mejores = elitismo)

4. RETORNAR el mejor individuo encontrado

Algoritmos Genéticos

SELECCIÓN: Las mejores soluciones deberían tener mayor probabilidad de reproducirse

- **Método de la ruleta** (proporcional al fitness)

$$P(\text{elegir } w_i) = \frac{L(w_i)}{\sum_j L(w_j)}$$

Pero ojo, si un individuo tiene fitness muy alto, domina la población → pérdida de diversidad

- **Selección por torneo** (en la práctica es más usada)

Número k muy grande → más presión selectiva

Su ventaja es que no requiere normalizar el fitness

Elegimos k individuos al azar y el mejor de todos esos será el “padre” seleccionado

- **Método del rango** (elitismo)

Los N_e mejores individuos de la generación actual pasan directamente a la siguiente, sin modificaciones

Garantiza que la mejor solución nunca se pierde

Riesgo: puede reducir la diversidad si N_e es grande

Algoritmos Genéticos

CRUCE: Combinar genética de dos padres para producir hijos que hereden características

Cruce de 1 punto

Padre 1: [A B C | D E F G]

Padre 2: [a b c | d e f g]

↑ punto de corte aleatorio

Hijo 1: [A B C | d e f g]

Hijo 2: [a b c | D E F G]

Cruce uniforme

Para cada posición elijo
aleatoriamente de cuál padre herdar

Padre 1: [A B C D E F]

Padre 2: [a b c d e f]

Hijo 1: [A b C d E f] ← mezcla aleatoria gen a gen

Cruce de 2 puntos

Padre 1: [A B | C D E | F G]

Padre 2: [a b | c d e | f g]

Hijo 1: [A B | c d e | F G]

Algoritmos Genéticos

MUTACIÓN: Introducir variabilidad aleatoria para evitar la convergencia prematura y explorar zonas no alcanzables por cruce solo

- **Mutación binaria** (bit flip)

Cromosoma original: [1 0 1 1 0 0 1 0]

Luego de la mutación: [1 0 1 0 0 0 1 0]

Lo estándar es que la probabilidad de mutación de un gen sea: $p_m = \frac{1}{\text{longitud cromosoma}}$

- **Mutación Real** (Gaussiana)

$$w' = w + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2)$$

La tasa de mutación debe ser baja ¿Por qué?

- Alta mutación → búsqueda casi aleatoria → se pierde lo aprendido.
- Baja mutación → permite que el cruce haga el trabajo, la mutación solo "repara" pérdidas de diversidad.

Algoritmos Genéticos

¿Qué rol cumple cada una de estas operaciones descriptas?

Operador	Rol en la búsqueda
Selección	EXPLOTACIÓN: presiona hacia mejores soluciones ya conocidas
Cruce	EXPLORACIÓN + EXPLOTACIÓN: combina bloques de soluciones buenas
Mutación	EXPLORACIÓN: Introduce novedad, evita convergencia prematura

Trade off:

- Selección fuerte + poca mutación → convergencia rápida pero riesgo de mínimo local
- Selección débil + mucha mutación → exploración amplia pero convergencia lenta

Los Algoritmos Genéticos balancean esta tensión automáticamente a través de la estructura de la población

Algoritmos Genéticos

Parámetros a tener en cuenta y sus efectos:

Parámetro	Rango típico	Efecto si es muy bajo	Efecto si es muy alto
Tamaño poblacional N	50 – 500	Poca diversidad, convergencia prematura	Costo computacional alto
Tasa de cruce p_c	0.6 – 0.9	Las generaciones cambian poco	Destruye individuos buenos
Tasa de mutación p_m	0.001 – 0.1	Convergencia prematura	Búsqueda casi aleatoria
Elitismo N_e	1 – 5% de N	Puede perder la mejor solución	Reduce diversidad
Número de generaciones	100 – 10000	No converge	Tiempo excesivo

Algoritmos Genéticos

¿Cuándo detener el algoritmo?

- Criterios habituales:
 1. Se alcanzó un número máximo de generaciones
 2. El fitness del mejor individuo no mejora en k generaciones consecutivas (estancamiento)
 3. La diversidad de la población cae por debajo de un umbral (convergencia)
 4. Se alcanzó un valor objetivo conocido (si se conoce el óptimo)
- Señales de convergencia prematura:
 - Todos los individuos tienen cromosomas casi idénticos pero el fitness no es óptimo
 - El fitness mejora en las primeras generaciones pero se estanca temprano
- Soluciones:
 - Aumentar la tasa de mutación
 - Reducir la presión selectiva (torneo con k menor)
 - Inyectar individuos aleatorios nuevos ("inmigración")
 - Reiniciar con diversidad manteniendo los mejores (restart)



FIN